

组成原理课程第 二 次实报告

实验名称：数据运算：定点加法

学号： 2312331 姓名： 王一诺 班次： 张金老师

一、实验目的

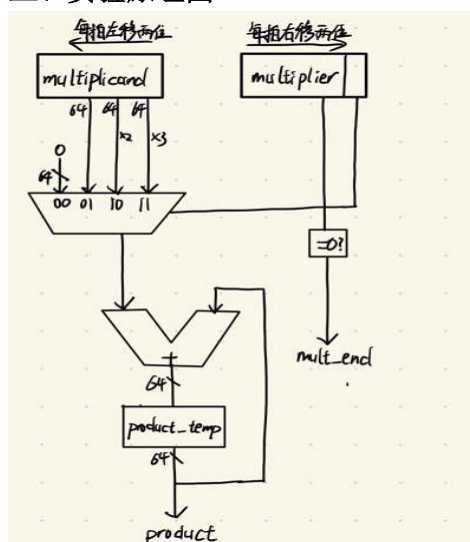
- 1.理解定点乘法的不同实现算法的原理，掌握基本实现算法。
- 2.熟悉并运用 verilog 语言进行电路设计。
- 3.为后续设计 cpu 的实验打下基础。

二、实验内容说明

针对组成原理第二次的乘法法器实验进行改进，要求：

- 1、将原有的迭代乘法改进成两位乘法，即每个时钟周期移位移两位，从而提高乘法效率。
- 2、将改进后的乘法器进行仿真验证。
- 3、将改进后的乘法器进行上实验箱验证，上箱验证时调整数据不在前 4 格显示。
- 4、实验报告中的原理图为迭代乘法的算法图，不再是顶层模块图。

三、实验原理图



总时钟节拍数与每拍移动的位数相关。时钟节拍缩短到 16 个，意味着要将每拍移动数从 1 位增至 2 位。此时，乘数的最后两位是我们需要的，因此数据选择器变成了四路数据选择器，分别对应着将被乘数乘上 0、1、2、3 再加到 product_temp 上。

四、实验步骤

Multiply.v 的修改

本部分实现了输入两个数并相乘的操作，要将迭代乘法改为两位乘法，缩短时钟周期，只需更改该模块代码即可。display、tb 和约束文件不必修改。接下来是对迭代乘法的具体改动：

(1) 修改被乘数位数移动：原代码为将被乘数的前 62 位取出，再将最低位的一位设置成 0，完成一位的左移。故，改成取出前 61 位再将最后两位设置成 0 就完成了两位的左移。如左下图所示。

//加载被乘数，运算时每次左移两位

```
reg [63:0] multiplicand;
always @ (posedge clk)
begin
    if (mult_valid)
    begin // 如果正在进行乘法，则被乘数每时钟左移两位
        multiplicand <= {multiplicand[61:0],2'b00};
    end
    else if (mult_begin)
    begin // 乘法开始，加载被乘数，为乘数1的绝对值
        multiplicand <= {32'd0,op1_absolute};
    end
end
end
```

//加载乘数，运算时每次右移两位

```
reg [31:0] multiplier;
always @ (posedge clk)
begin
    if (mult_valid)
    begin // 如果正在进行乘法，则乘数每时钟右移一位
        multiplier <= {2'b00,multiplier[31:2]};
    end
    else if (mult_begin)
    begin // 乘法开始，加载乘数，为乘数2的绝对值
        multiplier <= op2_absolute;
    end
end
end
```

(2) 修改乘数位移动：同理，改为取出第 31 位到第 2 位再将最高位的两位设置成 0 就实现了两位的右移。如右上图所示。

(3) 修改选择器：移动一位数字时，只有 0 和 1 两种情况。现在，每拍移动两位，需要考虑 0,1,2,3 即 00,01,10,11 这四种情况，也就要采用四路选择器，用两个 product 来记录一拍内乘法产生的增值。如下图所示。首先，根据乘数最低的一位给 partial_product1 赋值，若乘数第 0 位是 0 赋为 0，是 1 则赋为被乘数的 1 倍。然后，根据乘数第二低的那位对 partial_product2 赋值，若为 0 就赋为 0，是 1 则赋为被乘数左移一位后的值，也就是把被乘数的 2 倍赋值给了 partial_product2。

// 部分积

```
wire [63:0] partial_product1;
wire [63:0] partial_product2;
assign partial_product1 = multiplier[0] ? multiplicand : 64'd0;
assign partial_product2 = multiplier[1] ? {multiplicand[62:0],1'b0}:64'd0;
```

(4) 修改赋值句：最后将 temp 与两个 product 的和赋给 product_temp 即可。

```
product_temp <= product_temp + partial_product1+partial_product2;
```

五、实验结果分析

1. 仿真验证

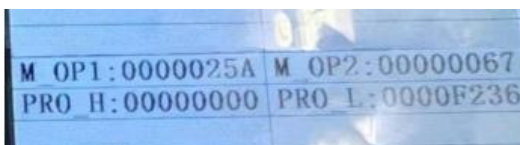
Name	Value	455,891 ps	455,892 ps	455,893 ps	455,894 ps	455,895 ps	455,896 ps	455,897 ps	455,898 ps
clk	1								
mult_begin	1								
> mult_op1[31:0]	4369						4369		
> mult_op2[31:0]	4369						4369		
> product[63:0]	19088161						19088161		
mult_end	0								

如上图所示，4369 和 4369 相乘，结果为 19088161，仿真结果是正确的。

Name	Value	460,480 ps	460,481 ps	460,482 ps	460,483 ps	460,484 ps	460,485 ps	460,486 ps	460,487 ps
clk	0								
mult_begin	1								
> mult_op1[31:0]	4369						4369		
> mult_op2[31:0]	8738						8738		
> product[63:0]	38176322						38176322		
mult_end	0								

再测一例，4369 与 8738 相乘，结果为 38176322，仿真结果是正确的。

2. 实验箱验证



602 (25A) 与 103 (67) 相乘，结果是 62006 (F236)，验证正确。

六、总结感想

1. 认识和学习了移位操作，对乘法器的计算原理和过程有了清晰的认知，并且思考和学习了通过增加移位长度改变乘法器运算速率的方法。

2. 对时钟周期的优化，联想到改变每拍运算位数，进而思及移位步长的变化，而改变移位位数又要考虑到中间结果的位数变化。整个思考过程执果索因，由表及内，而且要考虑全面，锻炼了思考问题的能力。